Information Sciences 576 (2021) 37–53

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Affine invariance of *meta*-heuristic algorithms

ZhongQuan Jian, GuangYu Zhu *

School of Mechanical Engineering and Automation, Fuzhou University, Fuzhou, PR China

ARTICLE INFO

Article history:
Received 6 November 2020
Received in revised form 14 June 2021
Accepted 20 June 2021
Available online 24 June 2021

Keywords:
Particle swarm optimization
Differential evolution
Optimal Foraging Algorithm
Coordinate system
Affine invariance
Meta-heuristic algorithms

ABSTRACT

An algorithm whose performance depends on the objective function being aligned with a privileged coordinate system is a poor choice in general because it is unlikely that the optimal orientation will be known in advance. In this paper, a property of *meta*-heuristic algorithms, named affine invariance, is introduced to verify whether the algorithm is depended on the privileged coordinate system or not. The concept of affine invariance is described in detail, and some classical algorithms, efficient in most test and actual problems, are proved to be affine invariant. While some recent algorithms in the literature are proved to be not affine invariant. As a conclusion, particle swarm optimization (PSO), differential evolution (DE) and optimal foraging algorithm (OFA) are affine invariant, while grey wolf optimizer (GWO), sine cosine algorithm (SCA) and butterfly optimization algorithm (BOA) are not affine invariant. Furthermore, comparison tests are designed to support the theoretical analysis results. In these tests, same random numbers and initial population are used to avoid the influence of randomness, thus, the conclusion is reliable.

1. Introduction

Thanks to mathematicians, real-world problems, specific and various, can be formulized in mathematics, and solved by mathematical methods. However, the traditional mathematical methods can't get a satisfactory solution when the objective functions are complex and high-dimensional. As a result, many problems have to be solved by trial and errors using various optimization techniques [1–2].

Without a loss of generality, a minimization optimal problem can be defined as [3]:

$$\text{Minimize} f(\boldsymbol{x}), \boldsymbol{x} = (x_1, x_2, \cdots, x_D) \in R^D, x_i^L \le x_i \le x_i^U \tag{1}$$

where f denotes the objective function. $\boldsymbol{x}$ is the decision vector of D design variables. x_i is the i-th component of $\boldsymbol{x}$, x_i^L and x_i^U are the lower and upper bounds of x_i, respectively. D is the number of dimensions. R^D represents the search space. Array variables are highlighted in bold face throughout this paper.

In recent years, the swarm intelligence and bio-inspired algorithms form a hot topic in the developments of new algorithms inspired by nature. Some algorithms have proved to be very efficient and have become popular tools for solving real-world problems [4]. The classical algorithms in the literature contain artificial bee colony (ABC) [5], cuckoo search (CS) [6], differential evolution (DE) [7–10], firefly algorithm (FA) [11], genetic algorithm (GA) [12], particle swarm optimization (PSO) [13,14]. Optimal foraging algorithm (OFA) [15], grey wolf optimizer (GWO) [16], sine cosine algorithm (SCA) [17] and butterfly optimization algorithm (BOA) [18] are new members of the family of *meta*-heuristics.

* Corresponding author at: Qi Shan Campus of Fuzhou University, No.2 Xue Yuan Road, Fuzhou City, Fujian Province, PR China.
E-mail addresses: 1521605374@qq.com (Z. Jian), zhugy@fzu.edu.cn (G. Zhu).

https://doi.org/10.1016/j.ins.2021.06.062
0020-0255/

In the last years, the number of bio-inspired or natural-inspired optimization algorithms in the literature has grown considerably, reaching unprecedented levels. However, there is no significant breakthrough in solving the optimization problems, which dark the future prospects of this field of research [19]. As we know, there is no algorithm in the literature that can solve all kinds of optimization problems well. Different algorithms with different ideas have their own advantages and disadvantages. It is interesting that one algorithm can solve a particular optimization problem very well, but another algorithm of the same type cannot get the same result when solving this optimization problem. And, one algorithm can solve an optimization problem very well, but cannot solve another optimization problem of the same type. Therefore, we conjecture that whether some algorithms were flawed?

Invariance is a strong non-empirical statement about generalization, which is important for the evaluation of search algorithms [20]. In physics, invariance, referred to as symmetry, is a fundamental concept in science. Invariance properties of a search algorithm denote identical behavior on a set of objective functions. Invariances are highly desirable: they imply uniform performance on classes of functions and therefore allow for generalization of empirical results. Translation invariance should be taken for granted in continuous domain optimization. Further invariances to linear transformations of the search space are desirable. Invariance should be a fundamental design criterion for any search algorithm [21].

The research of invariance in evolutionary algorithms has increasingly attracted to researches. In [21], CMA-ES exhibits the invariance against angle preserving transformations of the search space (rotation, reflection, and translation) if the initial search point(s) are transformed accordingly; furthermore, CMA-ES keeps the invariance against any invertible linear transformation of the search space. In [22], PSO with the linear velocity update rule is scale and frame invariant, while PSO with the classical velocity update rule lacks rotational invariance. In [23], SPSO2011 is proven to be invariant from rotation, scaling and translation. IPSO has been proven to be scaling and translation invariant while it is not rotation invariant in [22,24]. As observed in [25,26], the original mutation of DE, so-called DE/rand/1, is inherently rotational invariant since it is obtained as the weighted sum of one or more candidate solutions, while the crossover of DE is not rotation invariant.

In the process of proposing a new algorithm, it is impossible to use all the optimization problems to test the performance of the algorithm, so it is possible that the proposed algorithm exists defects, which are hard to be found. Hence, it is important to pay more attention to the possible flaws, e.g. invariance property, in the algorithms. In this paper, we consider the invariance properties of algorithms from the perspective of decision space.

An algorithm whose performance depends on the objective function being aligned with a privileged coordinate system is a poor choice in general because it is unlikely that the optimal orientation will be known in advance [26]. Therefore, an excellent algorithm should be independent of the coordinate system. To achieve this goal, affine invariance, contains affine scale, rotation, translation and general invariance, is defined to judge whether the algorithm relies on the privileged coordinate system or not.

According to linear algebra, a coordinate system corresponds to an affine space in the finite dimensional space [27–29]. In this paper, the original space is agreed to express the space with cartesian coordinate system, and the affine space is agreed to express the space with other coordinate system. In fact, the original space is just a special case of affine space. Any two affine spaces can convert to each other by affine transformation. Affine invariance is used to determine whether the performances of the algorithm are the same in different affine spaces (different coordinate systems).

Different from the definition of invariance in the literature, the affine invariance in this paper is the property under the affine space. In other words, affine invariance exists only when the algorithm executes spatial operation, e.g. the mutation operation of DE. One thing that spatial operations have in common is that operators can be expressed by explicit formulas, e.g. the update formula of DE/rand/1 [8]. And explicit formula has same form in different affine spaces, which is the theoretical basis of affine invariance. In our study, algorithms with explicit update formulas, such as PSO, OFA, DE, GWO, SCA, BOA, are concerned.

Note that the aim of this paper is to understand potential theoretical issues of the *meta*-heuristic algorithms and is not try to improve the algorithms to find better solutions. The contributions of this paper are summarized as follows:

1) The concept of invariance in search and optimization is reviewed. And, the concept and verification method of affine invariance are presented.
2) Two classical algorithms and four newest algorithms are employed to analyze their affine invariances theoretically and practically.
3) For each algorithm, multiple experimental comparisons are executed to verify its affine invariance.

The rest of this paper is organized as: the definition of affine invariance and algorithm affine invariance analysis are described detail in Section II. The experimental design is shown in Section III. In Section IV, the experimental study is carried out and discussed. The whole paper is discussed in Section V. Finally, the conclusion is summarized in Section VI.

2. Affine invariance analysis

2.1. Invariance review

In science, we desire a physical law or biological model to be invariant to environmental parameters, say weekday, temperature, or air humidity. The more invariance properties a model reveals, the wider is its applicability and the greater is its predictive power. The same idea holds for invariance properties of search algorithms. In search, invariance properties induce equivalence classes of objective functions, on which the performance of the search algorithm is identical [20].

In reference [20], a formal definition of invariance is given:

Definition. *Let $\mathscr{T} \subset \{T : R \to R\}$, $\mathscr{U} \subset \{U : R^n \to R^n\}$, where $\mathscr{T}$ and $\mathscr{U}$ contain the identity. Let S be the state space of the search algorithm, $s \in S$ and $\mathscr{A}_f : S \to S$ an iteration step of the algorithm under objective function f. The algorithm $\mathscr{A}$ is invariant under $\mathscr{T}$ and $\mathscr{U}$ if for all $T \in \mathscr{T}$, $U \in \mathscr{U}$ there exists a bijective state space transformation $J_{T,U} : S \to S$ such that for all $f \in R^n \to R$, $s \in S$:*

$$J^{\circ}_{T,U}\mathscr{A}_{T\circ f\circ U}(s) = \mathscr{A}^{\circ}_{f}J_{T,U}(s) \tag{2}$$

or equivalently

$$\mathscr{A}_{T\circ f\circ U}(s) = J^{-1}_{T,U}\circ\mathscr{A}^{\circ}_{f}J_{T,U}(s) \tag{3}$$

For randomized algorithms, equality (3) holds almost surely, given appropriately coupled random number realizations, otherwise in distribution. The set of functions $\{T\circ f\circ U | T \in \mathscr{T}, U \in \mathscr{U}\}$ is an invariance set of f for algorithm $\mathscr{A}$. The notation $\circ$ represents the composite mapping.

The equality (3) illustrates that for any function $f : R^n \to R$, $T \in \mathscr{T}$, $U \in \mathscr{U}$, the algorithm $\mathscr{A}$ optimizes the function $T\circ f\circ U$ with initial state s just like the function f with initial state $J_{T,U}(s)$. By reducing $\mathscr{T}$ to identity transformation, one can consider invariance under $\mathscr{U} \subset \{U : R^n \to R^n\}$ solely. The invariance will be concerned under search space transformation U, and the invariance properties is described as the equivalence of two functions f and $f\circ U$.

Based on the above definition, the invariance of the linear and the classical velocity update rules of PSO are investigated in [22]. The proofs of the scale and translation invariance of both the linear and the classical velocity update rules are presented. In addition, the linear velocity update rule is proved to be invariant of rotation, while the classical velocity update rule is proved to be not rotationally invariant. More studies about the invariance of PSO are mentioned in [20,23,24].

The invariance of DE is also being studied. As mentioned in [25], mutation in DE is inherently rotational invariant since it simultaneously perturbs all the variables. On the other hand, crossover, albeit fundamental for achieving a good performance, retains some of the variables, thus is not rotational invariant. A rotational invariant DE, DE/current-to-rand/1, is introduced in [25] to compare with the standard DE. Numerical results indicate that the rotational invariant DE/current-to-rand/1 is systematically outperformed by the standard DE.

Although PSO and DE are not invariant, sufficient research and experiments show that PSO and DE are excellent and effective search algorithms [19], which seems to indicate that invariance is not a sufficient condition for good algorithms. Furthermore, GA with crossover and mutation operators are inherently not invariant. So, is invariance not a critical property of algorithms?

In our opinion, the invariance concept of algorithms is too strict. According the definition of invariance, an algorithm is invariant means that it has the same performance for the same kind of functions. And two functions of the same kind can be transformed to each other by coordinate transformation. Coordinate transformation is spatial operator in mathematics. Hence, we believe that invariant algorithm only need to remain invariant during the spatial operations.

2.2. Affine space

In this paper, the research work is focused on the solutions of optimization problems. According to formula (1), a solution of the objective function can be described as a D-tuple intuitively $\boldsymbol{x} = (x_1, x_2, \cdots, x_D)$, which can be viewed as a vector in the vector space with the x_i as its component, also called coordinate under the nature basis $\boldsymbol{B_e} = \{\boldsymbol{e}^1, \boldsymbol{e}^2, \cdots, \boldsymbol{e}^{\boldsymbol{D}}\}$, where $\boldsymbol{e}^{\boldsymbol{k}}$ is a vector of length D composed of all zeros and a 1 in the k-th design variable [27–33]. For a D-dimensional space, the natural basis and the origin $\boldsymbol{O}$ constitute the spatial cartesian coordinate system [29]. Therefore, the solution $\boldsymbol{x}$ is also represented a point in this cartesian coordinate system.

In terms of basic mathematical concepts, there are innumerable coordinate systems in a finite dimensional space, and a coordinate system corresponds to an affine space. In order to avoid confusion, the space based on the cartesian coordinate system is referred to original space, and the space based on other coordinate system is called affine space. There is no doubt that the original space is a kind of affine space.

For any two different affine spaces in a finite dimensional space, they can convert to each other by the affine transformation. In other words, different coordinate systems can transform to each other by the affine transformation [34]. The affine transformation can be given by a matrix A and a vector $\boldsymbol{b}$.

$$g = \boldsymbol{x}\boldsymbol{A} + \boldsymbol{b} : R^D \rightarrow R^D \tag{4}$$

where $\boldsymbol{A}$ is the basis transformation matrix, $\boldsymbol{b}$ is the translation vector of origin. If the matrix $\boldsymbol{A}$ is invertible, then g is a bijective function.

If $\boldsymbol{b}$ is the null vector, and $\boldsymbol{A}$ is the covariance matrix of the problem, g represents a transformation from the coordinate system based on the natural basis to the coordinate system based on the eigenvectors of the covariance matrix of the problem. As shown in [31–33], by using this method, the problem space is transformed into a space that can decompose the problem based on its basis, and the directions that diagonalise the covariance matrix are chose to improve the performance of the pattern search.

Let $\boldsymbol{x}$ denotes a point in the original space, and there exist a point $\boldsymbol{x}'$ in an affine space, satisfying:

$$\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{A} + \boldsymbol{b} \tag{5}$$

where $\boldsymbol{x}$ and $\boldsymbol{x}'$ are the same point in the finite space, but they have different representations due to different coordinate systems.

Let $f_o(\boldsymbol{x})$ denotes the objective function in the original space, and $f_a(\boldsymbol{x}')$ denotes the objective function in the affine space. According to equality (3), it can be concluded that:

$$f_a(\boldsymbol{x}') = f_o(\boldsymbol{x}) \tag{6}$$

Since matrix $\boldsymbol{A}$ is invertible, formula (5) can be expressed as:

$$f_a(\boldsymbol{x}') = f_o\left((\boldsymbol{x}' - \boldsymbol{b})\boldsymbol{A}^{-1}\right) \tag{7}$$

Formula (7) tells us that solving $f_a(\boldsymbol{x}')$ in the affine space is equivalent to solving $f_o\left((\boldsymbol{x} - \boldsymbol{b})\boldsymbol{A}^{-1}\right)$ in the original space. It also provides the way to calculate the fitness of $\boldsymbol{x}'$ when the function f_a is unknow.

Due to the different choice of matrix $\boldsymbol{A}$ and vector $\boldsymbol{b}$, affine transformation can be roughly divided into four kinds: scale transformation, translation transformation, rotation transformation and general transformation. Among them, translation transformation can clearly distinguish whether the algorithm is sensitive to the presentation of the optimal solution. In translation transformation, the matrix $\boldsymbol{A}$ is identity, and the vector $\boldsymbol{b}$ is nonzero. Then, formula (7) can write as $f_a(\boldsymbol{x}') = f_o(\boldsymbol{x}' - \boldsymbol{b})$, it means that solving $f_a(\boldsymbol{x}')$ in the affine space is equal to solving $f_o(\boldsymbol{x} - \boldsymbol{b})$ in the original space. Therefore, an algorithm is sensitive to the presentation of the solution if it has different performances on $f(\boldsymbol{x})$ and $f(\boldsymbol{x} - \boldsymbol{b})$.

2.3. Algorithm execution process

Algorithm 1 summarizes the execution process of the algorithm in the original space and affine space. The execution processes of the algorithm in the original space and affine space are the same, which include the following procedures: firstly, the initial population is generated randomly in the constraint space; next, the candidates are obtained with the update formula, and their fitness are calculated; after that, a new population is obtained according to certain rules for the next iteration; repeat the process above until the termination condition is met; finally, output the final result.

Algorithm 1

Algorithm executes in the original space

Input: population size N, maximum iterations t_{max}, optimization function f_o, initial population $\boldsymbol{X}^{\boldsymbol{t}=0} = \{\boldsymbol{x}_1^{\boldsymbol{t}}, \boldsymbol{x}_2^{\boldsymbol{t}}, \cdots, \boldsymbol{x}_N^{\boldsymbol{t}}\}$, algorithm update rules (including algorithm parameters).**Output:** final optimization results $\boldsymbol{X}^{\boldsymbol{t}_{max}}, \boldsymbol{F}^{\boldsymbol{t}_{max}}$.1 **while** $t \leq t_{max}$ **do**2 The candidate population $\boldsymbol{X}^{\boldsymbol{t}+1}$ is generated by the algorithm update rules.3 The fitness values of the candidates are calaulated by $\boldsymbol{F}^{\boldsymbol{t}+1} = \{f_o(\boldsymbol{x}_1^{\boldsymbol{t}+1}), \cdots, f_o(\boldsymbol{x}_N^{\boldsymbol{t}+1})\}$.4 A new population $\boldsymbol{Y}$ with its fitness vector $\boldsymbol{V}$ is construct by picking proper individuals from $\boldsymbol{X}^{\boldsymbol{t}+1}$ and $\boldsymbol{X}^{\boldsymbol{t}}$.5 setting $t = t + 1$; $\boldsymbol{X}^{\boldsymbol{t}} = \boldsymbol{Y}, \boldsymbol{F}^{\boldsymbol{t}} = \boldsymbol{V}$.6 **end while**

Algorithm executes in the affine space

Input: population size N, maximum iterations t_{max}, optimization function f_o, transformation matrix $\boldsymbol{A}$, translation vector $\boldsymbol{b}$, initial population $\boldsymbol{X}'^{\boldsymbol{t}=0} = \boldsymbol{X}^{\boldsymbol{t}}\boldsymbol{A} + \boldsymbol{b}$, algorithm update rules (including algorithm parameters). **Output:** final optimization results $\boldsymbol{X}'^{\boldsymbol{t}_{max}}, \boldsymbol{F}'^{\boldsymbol{t}_{max}}$.1 **while** $t \leq t_{max}$ **do**2 The candidate population $\boldsymbol{X}'^{\boldsymbol{t}+1}$ is generated by the algorithm update rules.3 The fitness values of the candidates are calaulated by $\boldsymbol{F}'^{\boldsymbol{t}+1} = \left\{f_o\left((\boldsymbol{x}_1'^{\boldsymbol{t}+1} - \boldsymbol{b})\boldsymbol{A}^{-1}\right), \cdots, f_o\left((\boldsymbol{x}_N'^{\boldsymbol{t}+1} - \boldsymbol{b})\boldsymbol{A}^{-1}\right)\right\}$.4 A new population $\boldsymbol{Y}'$ with its fitness vector $\boldsymbol{V}'$ is construct by picking proper individuals from $\boldsymbol{X}'^{\boldsymbol{t}+1}$ and $\boldsymbol{X}'^{\boldsymbol{t}}$.5 setting $t = t + 1$; $\boldsymbol{X}'^{\boldsymbol{t}} = \boldsymbol{Y}', \boldsymbol{F}'^{\boldsymbol{t}} = \boldsymbol{V}'$.6 **end while**

As shown in Algorithm 1, the implementations of the algorithm in the original space and affine space are different and related. The identical line numbers in both algorithms represent the same operation in different spaces, hence, the two algorithms (actually, they are the same algorithm executed in different spaces) are placed side by side for clearer comparison.

Firstly, in the **Input** stage, the initial population in the affine space is obtained by the affine transformation of the population in the original space. Secondly, the operations on line 2, 4 and 5 are the same in different spaces due to they are coordinate-independent operations, but the inputs and outputs of the operations in different spaces are different. Thirdly, the operations on line 3 of different spaces are different since they are coordinate-dependent. As shown in equalities (6) and (7), the fitness of the solutions in the affine space can be calculated using the function in the original space.

According to Algorithm 1, the output results of the algorithm in different spaces under the same inputs are obtained. Comparing the output results, whether the algorithm is affine invariant is determined.

2.4. Affine invariance

For an algorithm, its update operation is the most important, which directly determines the performance of the algorithm in solving optimization problems. The update operation independent of coordinate systems is a necessary requirement of a good algorithm. Hence, the definition of affine invariance is described as: there are two affine spaces (original space is a kind of affine space) in the D-dimensional space, they have different coordinate systems, shown in Fig. 1. For a point in D-dimensional space, there are different presentations in different affine spaces $\boldsymbol{x}$ and $\boldsymbol{x}'$, satisfying $\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{A} + \boldsymbol{b}$. Since the initial population in the affine space is a mapping of the initial population in the original space, $\boldsymbol{x}'^1 = \boldsymbol{x}^1\boldsymbol{A} + \boldsymbol{b}$ is true undoubtedly. Assuming that $\boldsymbol{x}'^t = \boldsymbol{x}^t\boldsymbol{A} + \boldsymbol{b}$ is true, after update operation, the algorithm is considered as affine invariant if $\boldsymbol{x}'^{t+1} = \boldsymbol{x}^{t+1}\boldsymbol{A} + \boldsymbol{b}$ is still true, which means that $\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{A} + \boldsymbol{b}$ is true in the whole iteration; otherwise, the algorithm is not affine invariant.

An algorithm is considered to be affine invariant if the update operation does not change the candidates' relationships between the affine space and the original space. It means that the optimization processes in the original space and affine space are consistent. In other word, they have the same optimal solution in each iteration. Although this optimal solution has different presentations in different affine space, but they are the same point essentially.

2.5. Update operation analysis

In this paper, six algorithms with explicit update formula in the literature are selected to analyze their affine invariance. As a result, PSO, OFA and DE are proved to be affine invariant, while GWO, SCA and BOA are not affine invariant. The mathematical affine invariance proofs are given in this section.

2.5.1. Affine invariance of PSO

The update operation of PSO is expressed as:

$$\begin{cases} \boldsymbol{v}^{j,t+1} = w\boldsymbol{v}^{j,t} + c_1 r_1(\boldsymbol{x}^{p,t} - \boldsymbol{x}^{j,t}) + c_2 r_2(\boldsymbol{x}^{g,t} - \boldsymbol{x}^{j,t}) \\ \boldsymbol{x}^{j,t+1} = \boldsymbol{x}^{j,t} + \boldsymbol{v}^{j,t+1} \end{cases} \tag{8}$$

where w, c_1, c_2 are determined parameters. r_1, r_2 are uniform random numbers. $\boldsymbol{x}^{p,t}$ denotes the best solution historically achieved by the particle itself, and $\boldsymbol{x}^{g,t}$ denotes the global best solution found currently. $\boldsymbol{v}^{j,t}$ the speed of particle.

Notice that $\boldsymbol{v}^{j,t}$ is a velocity vector, which has different search space relative to $\boldsymbol{x}^{j,t}$. It doesn't represent a point in space, then, it can't be mapped to the affine space. Thus, the first equality in formula (8) is put into the second equality, and then, the part that contain $\boldsymbol{v}^{j,t}$ is extracted. Hence, the update operation should be rewritten as:

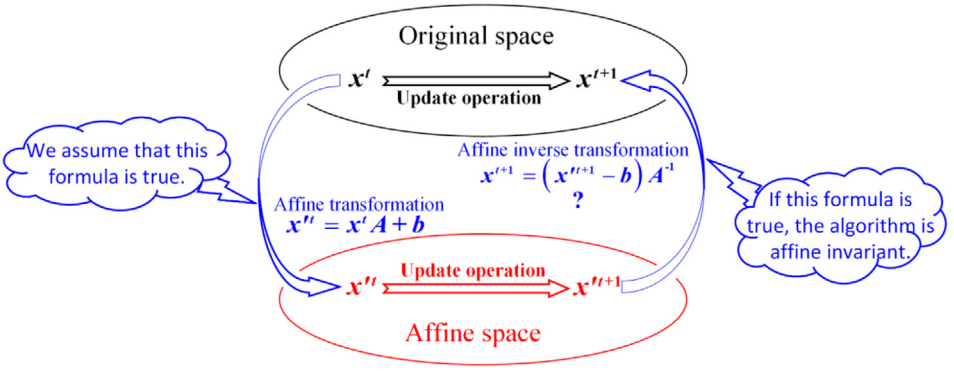

Fig. 1. The schematic diagram of the affine invariance of the algorithm.

$$\begin{cases} \boldsymbol{x}^{j,t+1} = \boldsymbol{x}^{j,t} + c_1 r_1(\boldsymbol{x}^{p,t} - \boldsymbol{x}^{j,t}) + c_2 r_2(\boldsymbol{x}^{g,t} - \boldsymbol{x}^{j,t}) \\ \boldsymbol{x}^{j,t+1} = \boldsymbol{x}^{j,t+1} + \boldsymbol{w}\boldsymbol{v}^{j,t} \\ \boldsymbol{v}^{j,t+1} = w\boldsymbol{v}^{j,t} + c_1 r_1(\boldsymbol{x}^{p,t} - \boldsymbol{x}^{j,t}) + c_2 r_2(\boldsymbol{x}^{g,t} - \boldsymbol{x}^{j,t}) \end{cases} \tag{9}$$

Formula (9) is equal to formula (8) in search, and the main update formula is:

$$\boldsymbol{x}^{j,t+1} = \boldsymbol{x}^{j,t} + c_1 r_1(\boldsymbol{x}^{p,t} - \boldsymbol{x}^{j,t}) + c_2 r_2(\boldsymbol{x}^{g,t} - \boldsymbol{x}^{j,t}) \tag{10}$$

Proof:. *The mapping solutions are* $\boldsymbol{x}^{j',t}$, $\boldsymbol{x}^{p',t}$, $\boldsymbol{x}^{g',t}$ *in the affine space. Then*:

$$\boldsymbol{x}^{j',t+1} = \boldsymbol{x}^{j',t} + c_1 r_1(\boldsymbol{x}^{p',t} - \boldsymbol{x}^{j',t}) + c_2 r_2(\boldsymbol{x}^{g',t} - \boldsymbol{x}^{j',t}) \tag{11}$$

According to formula (5) and (11), $(\boldsymbol{x}^{j',t+1} - \boldsymbol{b})\boldsymbol{A}^{-1}$ is obtained**:**

$$(\boldsymbol{x}^{j',t+1} - \boldsymbol{b})\boldsymbol{A}^{-1} = \boldsymbol{x}^{j,t} + c_1 r_1(\boldsymbol{x}^{p,t} - \boldsymbol{x}^{j,t}) + c_2 r_2(\boldsymbol{x}^{g,t} - \boldsymbol{x}^{j,t}) \tag{12}$$

From formula (12) and formula (10), it's obvious that $(\boldsymbol{x}^{j',t+1} - \boldsymbol{b})\boldsymbol{A}^{-1} = \boldsymbol{x}^{j,t+1}$, which means that PSO is affine invariant.

2.5.2. Affine invariance of OFA

The update operation of OFA is expressed as:

$$\boldsymbol{x}^{j,t+1} = \boldsymbol{x}^{j,t} + kr_1(\boldsymbol{x}^{s,t} - \boldsymbol{x}^{j,t}) - kr_2(\boldsymbol{x}^{s,t} - \boldsymbol{x}^{j,t}) \tag{13}$$

where k is the scale factor. r_1, r_2 are uniform random numbers. $\boldsymbol{x}^{s,t}$ denotes the recruited solution, satisfying $\boldsymbol{x}^{s,t}$ better than $\boldsymbol{x}^{j,t}$.

Proof:. *The corresponding solutions are* $\boldsymbol{x}^{j',t}$, $\boldsymbol{x}^{s',t}$ *in the affine space. Then*:

$$\boldsymbol{x}^{j',t+1} = \boldsymbol{x}^{j',t} + kr_1(\boldsymbol{x}^{s',t} - \boldsymbol{x}^{j',t}) - kr_2(\boldsymbol{x}^{s',t} - \boldsymbol{x}^{j',t}) \tag{14}$$

According to formula (5) and (14), $(\boldsymbol{x}^{j',t+1} - \boldsymbol{b})\boldsymbol{A}^{-1}$ is obtained:

$$(\boldsymbol{x}^{j',t+1} - \boldsymbol{b})\boldsymbol{A}^{-1} = \boldsymbol{x}^{j,t} + kr_1(\boldsymbol{x}^{s,t} - \boldsymbol{x}^{j,t}) + kr_2(\boldsymbol{x}^{s,t} - \boldsymbol{x}^{j,t}) \tag{15}$$

From formula (15) and formula (13), it's obvious that $(\boldsymbol{x}^{j',t+1} - \boldsymbol{b})\boldsymbol{A}^{-1} = \boldsymbol{x}^{j,t+1}$, which means that OFA is affine invariant.

2.5.3. Affine invariance of DE in special case

As we know, the crossover of DE will be ignored due to it is not executed in the affine space. Therefore, the mutation of DE is expressed as:

$$\boldsymbol{x}^{j,t+1} = \boldsymbol{x}^{l,t} + F(\boldsymbol{x}^{m,t} - \boldsymbol{x}^{n,t}) \tag{16}$$

where F is the weighting factor. $\boldsymbol{x}^{l,t}$, $\boldsymbol{x}^{m,t}$, $\boldsymbol{x}^{n,t}$ are selected solutions, which are different from $\boldsymbol{x}^{j,t}$.

Proof:. *The corresponding individuals are* $\boldsymbol{x}^{l',t}$, $\boldsymbol{x}^{m',t}$, $\boldsymbol{x}^{n',t}$ *in the affine space. Then*:

$$\boldsymbol{x}^{j',t+1} = \boldsymbol{x}^{l',t} + F(\boldsymbol{x}^{m',t} - \boldsymbol{x}^{n',t}) \tag{17}$$

According to formula (5) and (17), $(\boldsymbol{x}^{j',t+1} - \boldsymbol{b})\boldsymbol{A}^{-1}$ is obtained:

$$(\boldsymbol{x}^{j',t+1} - \boldsymbol{b})\boldsymbol{A}^{-1} = \boldsymbol{x}^{l,t} + F(\boldsymbol{x}^{m,t} - \boldsymbol{x}^{n,t})) \tag{18}$$

From formula (18) and formula (16), it's obvious that $(\boldsymbol{x}^{j',t+1} - \boldsymbol{b})\boldsymbol{A}^{-1} = \boldsymbol{x}^{j,t+1}$, which means that the mutation of DE is affine invariant.

2.5.4. Affine invariance of GWO

The update operation of GWO is expressed as:

$$\begin{cases} \boldsymbol{Y}^{\alpha,t} = \boldsymbol{x}^{\alpha,t} - F_{\alpha}^{A}\left|F_{\alpha}^{C}\boldsymbol{x}^{\alpha,t} - \boldsymbol{x}^{j,t}\right| \\ \boldsymbol{Y}^{\beta,t} = \boldsymbol{x}^{\beta,t} - F_{\beta}^{A}\left|F_{\beta}^{C}\boldsymbol{x}^{\beta,t} - \boldsymbol{x}^{j,t}\right| \\ \boldsymbol{Y}^{\gamma,t} = \boldsymbol{x}^{\gamma,t} - F_{\gamma}^{A}\left|F_{\gamma}^{C}\boldsymbol{x}^{\gamma,t} - \boldsymbol{x}^{j,t}\right| \\ \boldsymbol{x}^{j,t+1} = (\boldsymbol{Y}^{\alpha,t} + \boldsymbol{Y}^{\beta,t} + \boldsymbol{Y}^{\gamma,t})/3 \end{cases} \tag{19}$$

where F_{α}^{A}, F_{β}^{A}, F_{γ}^{A}, F_{α}^{C}, F_{β}^{C}, F_{γ}^{C} are the algorithm parameters. $\boldsymbol{x}^{\boldsymbol{\alpha},t}$, $\boldsymbol{x}^{\boldsymbol{\beta},t}$, $\boldsymbol{x}^{\boldsymbol{\gamma},t}$ denote alpha wolf, beta wolf and delta wolf, respectively.

Proof:. *The corresponding individuals are* $\boldsymbol{x}^{\boldsymbol{j'},t}$, $\boldsymbol{x}^{\boldsymbol{\alpha'},t}$, $\boldsymbol{x}^{\boldsymbol{\beta'},t}$, $\boldsymbol{x}^{\boldsymbol{\gamma'},t}$ *in the affine space. Then*:

$$\begin{cases} \boldsymbol{Y}^{\boldsymbol{\alpha'},t} = \boldsymbol{x}^{\boldsymbol{\alpha'},t} - F_{\alpha}^{A}\left|F_{\alpha}^{C}\boldsymbol{x}^{\boldsymbol{\alpha'},t} - \boldsymbol{x}^{\boldsymbol{j'},t}\right| \\ \boldsymbol{Y}^{\boldsymbol{\beta'},t} = \boldsymbol{x}^{\boldsymbol{\beta'},t} - F_{\beta}^{A}\left|F_{\beta}^{C}\boldsymbol{x}^{\boldsymbol{\beta'},t} - \boldsymbol{x}^{\boldsymbol{j'},t}\right| \\ \boldsymbol{Y}^{\boldsymbol{\gamma'},t} = \boldsymbol{x}^{\boldsymbol{\gamma'},t} - F_{\gamma}^{A}\left|F_{\gamma}^{C}\boldsymbol{x}^{\boldsymbol{\gamma'},t} - \boldsymbol{x}^{\boldsymbol{j'},t}\right| \\ \boldsymbol{x}^{\boldsymbol{j'},t+1} = \left(\boldsymbol{Y}^{\boldsymbol{\alpha'},t} + \boldsymbol{Y}^{\boldsymbol{\beta'},t} + \boldsymbol{Y}^{\boldsymbol{\gamma'},t}\right)/3 \end{cases} \tag{20}$$

According to formula (5) and (20), $(\boldsymbol{x}^{\boldsymbol{j'},t+1} - \boldsymbol{b})\boldsymbol{A}^{-1}$ is obtained:

$$\begin{cases} \left(\boldsymbol{Y}^{\boldsymbol{\alpha'},t} - \boldsymbol{b}\right)\boldsymbol{A}^{-1} = \boldsymbol{x}^{\boldsymbol{\alpha},t} - F_{\alpha}^{A}\left|\left(F_{\alpha}^{C}\boldsymbol{x}^{\boldsymbol{\alpha},t} - \boldsymbol{x}^{\boldsymbol{j},t}\right)\boldsymbol{A} + \left(1 - F_{\alpha}^{C}\right)\boldsymbol{b}\right|\boldsymbol{A}^{-1} \\ \left(\boldsymbol{Y}^{\boldsymbol{\beta'},t} - \boldsymbol{b}\right)\boldsymbol{A}^{-1} = \boldsymbol{x}^{\boldsymbol{\beta},t} - F_{\beta}^{A}\left|\left(F_{\beta}^{C}\boldsymbol{x}^{\boldsymbol{\beta},t} - \boldsymbol{x}^{\boldsymbol{j},t}\right)\boldsymbol{A} + \left(1 - F_{\beta}^{C}\right)\boldsymbol{b}\right|\boldsymbol{A}^{-1} \\ \left(\boldsymbol{Y}^{\boldsymbol{\gamma'},t} - \boldsymbol{b}\right)\boldsymbol{A}^{-1} = \boldsymbol{x}^{\boldsymbol{\gamma},t} - F_{\gamma}^{A}\left|\left(F_{\gamma}^{C}\boldsymbol{x}^{\boldsymbol{\gamma},t} - \boldsymbol{x}^{\boldsymbol{j},t}\right)\boldsymbol{A} + \left(1 - F_{\gamma}^{C}\right)\boldsymbol{b}\right|\boldsymbol{A}^{-1} \\ (\boldsymbol{x}^{\boldsymbol{j'},t+1} - \boldsymbol{b})\boldsymbol{A}^{-1} = \frac{\left(\left(\boldsymbol{Y}^{\boldsymbol{\alpha'},t} - \boldsymbol{b}\right)\boldsymbol{A}^{-1} + \left(\boldsymbol{Y}^{\boldsymbol{\beta'},t} - \boldsymbol{b}\right)\boldsymbol{A}^{-1} + \left(\boldsymbol{Y}^{\boldsymbol{\gamma'},t} - \boldsymbol{b}\right)\boldsymbol{A}^{-1}\right)}{3} \end{cases} \tag{21}$$

By comparing formula (21) with formula (19), it is obvious that formula (21) is quite different from formula (19). GWO is affine invariant if and only if $\boldsymbol{b}$ is zero vector and $\boldsymbol{A}$ is identity.

2.5.5. Affine invariance of SCA

The update operation of SCA is expressed as:

$$\boldsymbol{x}^{\boldsymbol{j},t+1} = \begin{cases} \boldsymbol{x}^{\boldsymbol{j},t} + r_1 sin(r_2)\left|r_3\boldsymbol{x}^{\boldsymbol{g},t} - \boldsymbol{x}^{\boldsymbol{j},t}\right|, ifr_4 < 0.5 \\ \boldsymbol{x}^{\boldsymbol{j},t} + r_1 \cos(r_2)\left|r_3\boldsymbol{x}^{\boldsymbol{g},t} - \boldsymbol{x}^{\boldsymbol{j},t}\right|, ifr_4 \geq 0.5 \end{cases} \tag{22}$$

where r_1, r_2, r_3, r_4 are uniform random numbers. $\boldsymbol{x}^{\boldsymbol{g},t}$ denotes the global best solution found currently.

Proof:. *The corresponding individuals are* $\boldsymbol{x}^{\boldsymbol{j'},t}$, $\boldsymbol{x}^{\boldsymbol{g'},t}$ *in the affine space. Then*:

$$\boldsymbol{x}^{\boldsymbol{j'},t+1} = \begin{cases} \boldsymbol{x}^{\boldsymbol{j'},t} + r_1 sin(r_2)\left|r_3\boldsymbol{x}^{\boldsymbol{g'},t} - \boldsymbol{x}^{\boldsymbol{j'},t}\right|, ifr_4 < 0.5 \\ \boldsymbol{x}^{\boldsymbol{j'},t} + r_1 \cos(r_2)\left|r_3\boldsymbol{x}^{\boldsymbol{g'},t} - \boldsymbol{x}^{\boldsymbol{j'},t}\right|, ifr_4 \geq 0.5 \end{cases} \tag{23}$$

According to formula (5) and (23), $(\boldsymbol{x}^{\boldsymbol{j'},t+1} - \boldsymbol{b})\boldsymbol{A}^{-1}$ is obtained:

$$\left(\boldsymbol{x}^{\boldsymbol{j}',t+1} - \boldsymbol{b}\right)\boldsymbol{A}^{-1} = \begin{cases} \boldsymbol{x}^{\boldsymbol{j},t} + r_1 sin(r_2)\left|(r_3\boldsymbol{x}^{\boldsymbol{g},t} - \boldsymbol{x}^{\boldsymbol{j},t})A + (1 - r_3)b\right|\boldsymbol{A}^{-1}, ifr_4 < 0.5 \\ \boldsymbol{x}^{\boldsymbol{j},t} + r_1 \cos(r_2)\left|(r_3\boldsymbol{x}^{\boldsymbol{g},t} - \boldsymbol{x}^{\boldsymbol{j},t})A + (1 - r_3)b\right|\boldsymbol{A}^{-1}, ifr_4 \geq 0.5 \end{cases} \tag{24}$$

By comparing formula (24) with formula (22), it is obvious that formula (24) is quite different from formula (22). SCA is affine invariant if and only if $\boldsymbol{b}$ is zero vector and $\boldsymbol{A}$ is identity.

2.5.6. Affine invariance of BOA

The update operation of BOA is expressed as:

$$\boldsymbol{x}^{\boldsymbol{j},t+1} = \begin{cases} \boldsymbol{x}^{\boldsymbol{j},t} + \left(r^2\boldsymbol{x}^{\boldsymbol{g},t} - \boldsymbol{x}^{\boldsymbol{j},t}\right) \times f_r, ifr < 0.8 \\ \boldsymbol{x}^{\boldsymbol{j},t} + \left(r^2\boldsymbol{x}^{\boldsymbol{m},t} - \boldsymbol{x}^{\boldsymbol{n},t}\right) \times f_r, otherwise \end{cases} \tag{25}$$

where f_r is an algorithm parameter. r is a uniform random number. $\boldsymbol{x}^{\boldsymbol{g},t}$ denotes the global best solution found currently. $\boldsymbol{x}^{\boldsymbol{m},t}$, $\boldsymbol{x}^{\boldsymbol{n},t}$ are two randomly selected solutions.

Proof:. *The mapping individuals are* $\boldsymbol{x}^{\boldsymbol{j'},t}$, $\boldsymbol{x}^{\boldsymbol{g'},t}$, $\boldsymbol{x}^{\boldsymbol{m'},t}$, $\boldsymbol{x}^{\boldsymbol{n'},t}$ *in the affine space. Then*:

$$\boldsymbol{x}^{\boldsymbol{j'},t+1} = \begin{cases} \boldsymbol{x}^{\boldsymbol{j'},t} + \left(r^2\boldsymbol{x}^{\boldsymbol{g'},t} - \boldsymbol{x}^{\boldsymbol{j'},t}\right) \times f_r, ifr < 0.8 \\ \boldsymbol{x}^{\boldsymbol{j'},t} + \left(r^2\boldsymbol{x}^{\boldsymbol{m'},t} - \boldsymbol{x}^{\boldsymbol{n'},t}\right) \times f_r, otherwise \end{cases} \tag{26}$$

According to formula (5) and (26), $(\boldsymbol{x}^{j',t+1}-\boldsymbol{b})\boldsymbol{A}^{-1}$ is obtained:

$$\left(\boldsymbol{x}^{j',t+1}-\boldsymbol{b}\right)\boldsymbol{A}^{-1}=\begin{cases}\boldsymbol{x}^{j,t}+\left(r^2\boldsymbol{x}^{g,t}-\boldsymbol{x}^{j,t}+(r^2-1)\boldsymbol{b}\boldsymbol{A}^{-1}\right)f_r, ifr<0.8\\ \boldsymbol{x}^{j,t}+\left(r^2\boldsymbol{x}^{m,t}-\boldsymbol{x}^{n,t}+(r^2-1)\boldsymbol{b}\boldsymbol{A}^{-1}\right)f_r, otherwise\end{cases} \tag{27}$$

By comparing formula (27) with formula (25), it can be found that formula (27) has an extra part: $f_r(r^2-1)\boldsymbol{b}\boldsymbol{A}^{-1}$, which means that BOA is affine invariant if and only if $\boldsymbol{b}$ is zero vector. Imagining, if each component of the best solution g_i^t is a big number, the ith dimensional search step size of BOA $(x_i^{t+1}-x_i^t=r^2g_i^t-x_i^t)$ always going to be a big number, which is bad for exploitation.

3. Experiment design

In order to verify the correctness of the conclusion that PSO, OFA and DE are affine invariant, while GWO, SCA and BOA are not affine invariant. Multiple experimental comparisons were designed and carried out. In this section, the experimental conditions are described detailed.

3.1. Affine transformations

According to the requirement of the mutual transformation between the original space and affine space, the affine transformations must be invertible, which are summarized into the following four categories [19].

1) Scale transformation

Scale transformation refers to scale the basis vectors of the coordinate system with different proportion. That is, $\boldsymbol{A}$ is a diagonal matrix, the entries on the diagonal are not 0. $\boldsymbol{b}$ is a zero vector.

2) Rotation transformation

Rotation transformation is defined under angle preserving, i.e. rigid linear transformation of the search. That is, $\boldsymbol{A}$ is an orthogonal matrix, and $\boldsymbol{b}$ is a null vector.

3) Translation transformation

Translation transformation is the translation of the origin of the coordinate system, which does not change the basis vectors. That is, $\boldsymbol{A}$ is identity matrix, and $\boldsymbol{b}$ is a nonzero vector.

4) General transformation

Make the coordinate system general, let it not have to be orthogonal. That is, $\boldsymbol{A}$ is an arbitrary invertible matrix, and $\boldsymbol{b}$ is an arbitrary vector.

Among these four kinds of affine transformation, the matrix condition numbers under two norms of rotation and translation transformation are always 1, the vectors $\boldsymbol{b}$ in rotation and scale transformation are zero vectors. The condition number of matrix $\boldsymbol{A}$ is equal to the product of the norm of $\boldsymbol{A}$ and the norm of $\boldsymbol{A}$ inverse, namely $cond(\boldsymbol{A})=\|\boldsymbol{A}\|\hat{\mathrm{A}}\cdot\|\boldsymbol{A}^{(}-1)\|$, which is used to judge whether the matrix is sick or not. The larger the conditional number, the sicker the matrix is [35–37].

3.2. Test functions

The affine invariance of each algorithm was verified on six basic test functions [15,38], six CEC2017 functions [39] and two large-scale functions [15]. The mathematical formula of these functions is described detail in the supplementary material. The basic information of these functions are summarized in Table 1.

As shown in Table 1, F1-F6 are six basic test functions with 30 dimensions. F7 and F8 are two large-scale functions with the same search space, dimension and optimal value. The only difference between them is the positions of optimal solutions. The optimal solution of F7 is a zero vector, while the optimal solution of F8 is not a zero vector. From another perspective, the optimal solution of F8 is a translation transformation of F7. F9-F14 are functions picked from CEC2017 test suite. Including a unimodal function (F9), a simple multimodal function (F10), two hybrid functions (F11 and F12) and two composition functions (F13 and F14). Their optimal solutions are all nonzero.

Table 1
Test Functions.

No	Name	Bound	D	x^*	f_{opt}
F1	Sphere	[-5.12,5.12]	30	$[0,0,\cdots,0]$	0
F2	Schwefel 2.22	[-10,10]	30	$[0,0,\cdots,0]$	0
F3	Schwefel 1.2	[-65,65]	30	$[0,0,\cdots,0]$	0
F4	Schwefel 2.21	[-100,100]	30	$[0,0,\cdots,0]$	0
F5	Quartic	[-1.28,1.28]	30	$[0,0,\cdots,0]$	0
F6	Ackley	[-2,2]	30	$[0,0,\cdots,0]$	0
F7	Elliptic Function	[-100,100]	300	$[0,0,\cdots,0]$	0
F8	Shifted Elliptic Function	[-100,100]	300	nonzero	0
F9	Shifted and Rotated Bent Cigar	[-100,100]	30	nonzero	100
F10	Shifted and Rotated Expanded Scaffer	[-100,100]	30	nonzero	600
F11	Hybrid 1(N = 3)	[-100,100]	30	nonzero	1100
F12	Hybrid 6(N = 4)	[-100,100]	30	nonzero	1600
F13	Composition 1(N = 3)	[-100,100]	30	nonzero	2100
F14	Composition 6(N = 5)	[-100,100]	30	nonzero	2600

4. Experimental study

Based on the experiment design in Section 3, multiple experimental comparisons were carried out in this section. For each algorithm, the results obtained in the original space are compared with the results obtained in the affine spaces, respectively. These affine spaces are obtained from original space by different affine transformations (A.T.), they are summarized in Table 2. In scale and general transformations, two kinds of transformation matrix condition number are set to compare the influence of condition number (C.N.) on experimental results. In translation and general transformations, two kinds of vector $\boldsymbol{b}$ with different value ranges are set to verify the influence of vector $\boldsymbol{b}$ on the experimental results.

According to the definition of affine invariance, in theory, the evolutionary processes in different affine spaces are the same if the algorithm is affine invariant. But in the actual calculation, there exists some deviations in the calculation result due to the overflow of data. These deviations become more obvious when the error of the calculated function is smaller. Although the final output results in different affine spaces may be different, but the differences between these output results won't be obvious for the affine invariant algorithms due to they possess the same performance in different affine spaces.

4.1. Parameters setting

In this paper, the parameters of algorithms are non-significant because they are only compared with themselves. Therefore, the parameters of each algorithm are as consistent as possible with the source article. For PSO, the scale factor $w = 1$, the coefficients $c_1 = c_2 = 2.05$. For OFA, the scale coefficient $k = 1$. For DE, the weighting factor $F = 1$ and the crossover constant $CR = 1$. There are no parameters needed to determine additional in GWO and SCA because they are adaptive. For BOA, define sensor modality c is 0.01, power exponent a is a constant 0.1 and the switch probability $p = 0.8$.

The population size $N = 100$, the maximum number of iterations $t_{max} = 2000$. For every run, all test algorithms share the same initial population, transformation matrix and translation vector, which are generated randomly. All experiments are conducted using MATLAB R2019a and executed on a 64-bit personal computer equipped with an Intel(R) Core(TM) i7-8700 CPU @ 3.20 GHz processor and an operation system Windows 10 with RAM 32.00 GB.

Table 2
Affine Transformations.

Affine Transformation	Translation Vector $\boldsymbol{b}$	Condition Number
Original	$\boldsymbol{b} = 0$	1
Rotation	$\boldsymbol{b} = 0$	1
Translation_1	$b_i \in [-1, 1]$	1
Translation_2	$b_i \in [-10^3, 10^3]$	1
Scale_1	$\boldsymbol{b} = 0$	1
Scale_2	$\boldsymbol{b} = 0$	1000
General_1	$b_i \in [-1, 1]$	1
General_2	$b_i \in [-1, 1]$	1000
General_3	$b_i \in [-10^3, 10^3]$	1
‘General_4	$b_i \in [-10^3, 10^3]$	1000

4.2. Numerical experiments and convergence curves

In the experiment, each function is run 100 times independently, and the average values of the results obtained by each algorithm are recorded, shown in Table 3 - Table 8. Additionally, the convergence curves of each algorithm on F7 are shown in Fig. 2.

Table 3 tabulates the experimental results obtained by PSO. As shown in Table 3, PSO has the same or similar results in all affine spaces on each function. The experimental results support the conclusion that PSO is affine invariant in Section 2.5.1. Table 4 represents the simulated results obtained by OFA. From Table 4, it can be found that OFA possesses the same performance in different affine spaces on each function. Therefore, the conclusion that OFA is affine invariant is further verified. Table 5 lists the computational results obtained by DE. Table 5 indicates that DE has the same or similar results in all affine spaces on each function. Thus, the computational results support the conclusion that DE is affine invariant.

The experimental results obtained by GWO are summarized in Table 6. As shown in Table 6, the results of scale transformation are the same or similar to the original on each function, which indicates that GWO is scale invariant. But results in other affine spaces are quite different from the original, which indicates that GWO is not rotation, translation, and general invariant. Therefore, the conclusion that GWO is not affine invariant is consistent with the conclusion shown in Section 2.5.4.

Table 3
Simulated results of particle swarm optimization (PSO).

Fun.	Affine Transformation									
	Original	Rotation	$Translation_1$	$Translation_2$	$Scale_1$	$Scale_2$	$General_1$	$General_2$	$General_3$	$General_4$
F1	2.62E-01	2.62E-01	2.62E-01	2.62E-01	2.62E-01	2.62E-01	2.62E-01	2.62E-01	2.62E-01	2.43E-05
F2	3.19E + 00	3.19E + 00	3.20E + 00	3.15E + 00	3.19E + 00	3.19E + 00	3.19E + 00	3.19E + 00	3.19E + 00	1.65E-02
F3	2.25E + 01	2.24E + 01	2.26E + 01	2.20E + 01	2.24E + 01	2.25E + 01	2.24E + 01	2.26E + 01	2.26E + 01	2.27E-01
F4	3.15E-01	3.09E-01	3.32E-01	1.90E-01	3.09E-01	3.27E-01	3.08E-01	3.42E-01	3.16E-01	5.60E-02
F5	1.27E-01	1.15E-01	1.40E-01	9.06E-02	1.13E-01	8.97E-02	1.43E-01	1.14E-01	1.43E-01	1.94E-02
F6	1.38E-01	1.37E-01	1.48E-01	6.90E-02	1.39E-01	1.46E-01	1.39E-01	1.44E-01	1.42E-01	3.08E-02
F7	8.09E + 07	7.87E + 07	8.11E + 07	6.42E + 07	7.94E + 07	8.20E + 07	8.00E + 07	7.98E + 07	8.30E + 07	7.05E + 06
F8	2.19E + 09	2.19E + 09	2.19E + 09	2.18E + 09	2.20E + 09	2.20E + 09	2.20E + 09	2.20E + 09	2.20E + 09	8.29E + 06
F9	1.24E + 08	1.24E + 08	1.24E + 08	1.24E + 08	1.24E + 08	1.24E + 08	1.24E + 08	1.24E + 08	1.24E + 08	3.23E + 04
F10	4.69E + 01	4.68E + 01	4.69E + 01	4.55E + 01	4.69E + 01	4.73E + 01	4.70E + 01	4.70E + 01	4.68E + 01	6.13E-01
F11	1.79E + 02	1.79E + 02	1.79E + 02	1.79E + 02	1.79E + 02	1.79E + 02	1.79E + 02	1.79E + 02	1.79E + 02	2.02E-01
F12	1.24E + 03	1.25E + 03	1.24E + 03	1.25E + 03	1.25E + 03	1.24E + 03	1.25E + 03	1.24E + 03	1.25E + 03	4.04E + 00
F13	4.08E + 02	4.08E + 02	4.08E + 02	4.07E + 02	4.08E + 02	4.08E + 02	4.08E + 02	4.08E + 02	4.09E + 02	7.02E-01
F14	3.68E + 03	3.67E + 03	3.73E + 03	3.72E + 03	3.74E + 03	3.75E + 03	3.75E + 03	3.76E + 03	3.74E + 03	2.88E + 01

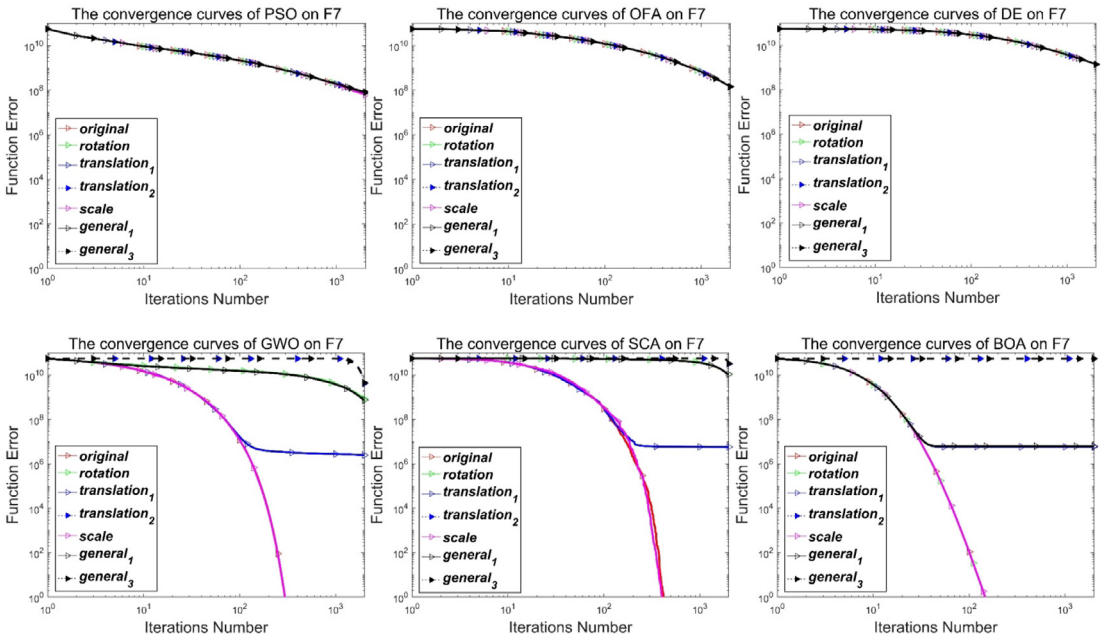

Fig. 2. The convergence curves of each algorithm on F7.

Table 4
Simulated Results of Optimal Foraging Algorithm (OFA).

Fun.	Affine Transformation									
	Original	Rotation	$Translation_1$	$Translation_2$	$Scale_1$	$Scale_2$	$General_1$	$General_2$	$General_3$	$General_4$
F1	2.82E-23	2.54E-23	2.64E-23	2.71E-23	2.82E-23	2.62E-23	2.60E-23	2.62E-23	2.77E-23	2.62E-23
F2	1.12E-14	1.09E-14	1.12E-14	0.00E + 00	1.12E-14	1.11E-14	1.20E-14	1.11E-14	7.77E-13	1.91E-14
F3	6.19E + 02	6.19E + 02	6.19E + 02	6.19E + 02	6.19E + 02	6.19E + 02	6.19E + 02	6.19E + 02	6.19E + 02	6.19E + 02
F4	1.53E + 00	1.49E + 00	1.53E + 00	1.48E + 00	1.53E + 00	1.53E + 00	1.53E + 00	1.51E + 00	1.50E + 00	1.53E + 00
F5	1.03E-02	1.09E-02	1.10E-02	1.08E-02	1.00E-02	1.06E-02	1.11E-02	1.03E-02	1.11E-02	1.09E-02
F6	3.13E-11	2.85E-11	2.99E-11	2.97E-11	3.13E-11	3.16E-11	3.12E-11	2.97E-11	3.10E-11	3.01E-11
F7	1.45E + 08	1.45E + 08	1.45E + 08	1.45E + 08	1.45E + 08	1.45E + 08	1.45E + 08	1.45E + 08	1.45E + 08	1.45E + 08
F8	1.28E + 08	1.28E + 08	1.28E + 08	1.28E + 08	1.28E + 08	1.28E + 08	1.28E + 08	1.28E + 08	1.28E + 08	1.28E + 08
F9	2.13E + 03	2.13E + 03	2.13E + 03	2.13E + 03	2.13E + 03	2.13E + 03	2.13E + 03	2.13E + 03	2.13E + 03	2.13E + 03
F10	8.32E-04	8.34E-04	8.32E-04	8.33E-04	8.32E-04	8.32E-04	8.37E-04	8.36E-04	8.28E-04	8.33E-04
F11	1.25E + 02	1.25E + 02	1.25E + 02	1.25E + 02	1.25E + 02	1.25E + 02	1.25E + 02	1.25E + 02	1.25E + 02	1.25E + 02
F12	1.44E + 03	1.44E + 03	1.44E + 03	1.44E + 03	1.44E + 03	1.44E + 03	1.44E + 03	1.44E + 03	1.44E + 03	1.44E + 03
F13	3.79E + 02	3.79E + 02	3.79E + 02	3.79E + 02	3.79E + 02	3.79E + 02	3.79E + 02	3.79E + 02	3.79E + 02	3.79E + 02
F14	2.29E + 03	2.29E + 03	2.29E + 03	2.29E + 03	2.29E + 03	2.29E + 03	2.29E + 03	2.29E + 03	2.29E + 03	2.29E + 03

Table 5
Simulated results of differential evolution (DE).

Fun.	Affine Transformation									
	Original	Rotation	$Translation_1$	$Translation_2$	$Scale_1$	$Scale_2$	$General_1$	$General_2$	$General_3$	$General_4$
F1	7.78E-10	8.05E-10	7.83E-10	7.69E-10	7.78E-10	7.94E-10	7.88E-10	8.03E-10	8.08E-10	8.09E-10
F2	2.86E-05	2.81E-05	2.84E-05	2.89E-05	2.86E-05	2.83E-05	2.85E-05	2.92E-05	2.89E-05	2.76E-05
F3	1.18E + 04	1.18E + 04	1.18E + 04	1.18E + 04	1.18E + 04	1.18E + 04	1.18E + 04	1.18E + 04	1.18E + 04	1.18E + 04
F4	1.67E + 01	1.67E + 01	1.68E + 01	1.68E + 01	1.67E + 01	1.67E + 01	1.65E + 01	1.65E + 01	1.68E + 01	1.66E + 01
F5	4.96E-02	4.89E-02	5.11E-02	5.09E-02	4.77E-02	4.91E-02	4.99E-02	5.04E-02	5.12E-02	5.01E-02
F6	2.83E-03	2.85E-03	2.81E-03	2.86E-03	2.83E-03	2.92E-03	2.90E-03	2.86E-03	2.82E-03	2.80E-03
F7	1.41E + 09	1.41E + 09	1.41E + 09	1.41E + 09	1.41E + 09	1.41E + 09	1.41E + 09	1.41E + 09	1.41E + 09	1.41E + 09
F8	7.29E + 08	7.29E + 08	7.29E + 08	7.29E + 08	7.29E + 08	7.29E + 08	7.30E + 08	7.29E + 08	7.29E + 08	7.30E + 08
F9	1.40E + 07	1.38E + 07	1.38E + 07	1.36E + 07	1.40E + 07	1.37E + 07	1.39E + 07	1.42E + 07	1.39E + 07	1.40E + 07
F10	1.01E-01	1.02E-01	1.02E-01	1.01E-01	1.01E-01	1.05E-01	1.01E-01	9.98E-02	1.01E-01	1.01E-01
F11	6.84E + 02	6.79E + 02	6.79E + 02	7.07E + 02	6.84E + 02	7.02E + 02	7.00E + 02	6.81E + 02	6.85E + 02	7.10E + 02
F12	9.15E + 02	9.18E + 02	9.02E + 02	9.28E + 02	9.15E + 02	9.32E + 02	9.21E + 02	8.94E + 02	9.39E + 02	9.11E + 02
F13	3.70E + 02	3.70E + 02	3.70E + 02	3.70E + 02	3.70E + 02	3.70E + 02	3.70E + 02	3.70E + 02	3.70E + 02	3.70E + 02
F14	2.54E + 03	2.54E + 03	2.54E + 03	2.54E + 03	2.54E + 03	2.54E + 03	2.54E + 03	2.54E + 03	2.54E + 03	2.54E + 03

Table 6
Simulated results of grey wolf optimizer (GWO).

Fun.	Affine Transformation									
	Original	Rotation	$Translation_1$	$Translation_2$	$Scale_1$	$Scale_2$	$General_1$	$General_2$	$General_3$	$General_4$
F1	1.5E-189	1.25E-83	1.87E-01	1.12E + 01	8.5E-189	3.4E-189	1.02E-03	4.39E-05	1.28E + 01	4.77E + 00
F2	1.1E-106	2.50E-04	8.26E-01	2.00E + 01	1.1E-106	5.8E-107	7.28E-02	1.40E-02	2.08E + 01	7.60E + 00
F3	3.07E-52	2.13E-02	1.73E + 00	2.32E + 03	7.59E-54	5.22E-52	2.24E-01	2.32E-02	2.97E + 03	7.77E + 02
F4	4.54E-43	4.24E-18	1.72E-01	1.30E + 01	3.44E-43	1.77E-43	9.57E-03	1.51E-03	9.14E + 00	1.60E + 00
F5	1.55E-04	1.45E-03	2.07E-01	5.98E + 00	1.51E-04	1.57E-04	6.35E-03	1.60E-03	7.60E + 00	2.05E + 00
F6	8.88E-16	5.14E-02	8.49E-01	1.99E + 01	8.88E-16	8.88E-16	3.11E-02	4.74E-02	2.00E + 01	1.22E + 01
F7	2.04E-34	8.51E + 08	2.51E + 06	4.28E + 09	1.83E-34	1.38E-34	7.53E + 08	7.67E + 08	4.53E + 09	9.51E + 08
F8	6.76E + 09	1.24E + 09	6.36E + 09	7.07E + 09	6.01E + 09	6.22E + 09	1.19E + 09	1.26E + 09	5.26E + 09	1.32E + 09
F9	7.57E + 08	1.02E + 07	8.73E + 08	2.47E + 09	8.40E + 08	7.90E + 08	1.11E + 07	9.81E + 06	7.59E + 08	6.16E + 07
F10	4.50E + 00	1.09E + 00	4.30E + 00	5.64E + 00	4.67E + 00	4.73E + 00	1.43E + 00	9.69E-01	4.96E + 00	1.68E + 00
F11	3.20E + 02	1.70E + 02	2.72E + 02	7.22E + 02	3.80E + 02	3.33E + 02	1.75E + 02	1.78E + 02	3.48E + 02	2.08E + 02
F12	7.81E + 02	6.65E + 02	7.73E + 02	1.06E + 03	7.44E + 02	7.89E + 02	6.96E + 02	6.66E + 02	9.90E + 02	8.19E + 02
F13	2.87E + 02	2.80E + 02	2.87E + 02	3.32E + 02	2.81E + 02	2.78E + 02	2.86E + 02	2.98E + 02	3.13E + 02	2.90E + 02
F14	1.92E + 03	1.70E + 03	1.94E + 03	2.27E + 03	1.92E + 03	1.90E + 03	1.69E + 03	1.73E + 03	2.07E + 03	1.74E + 03

The simulated results obtained by SCA are summarized in Table 7. As shown in Table 7, the results of scale transformation are the same or similar to the original on each function, which indicates that SCA is scale invariant. But, results in other affine spaces are quite different from the original, which indicates that SCA is not rotation, translation, and general invariant. Hence, the conclusion that SCA is not affine invariant is consistent with the conclusion shown in Section 2.5.5.

The computational results obtained by BOA are summarized in Table 8. As shown in Table 8, the results of rotation and scale transformation are the same or similar to the original on each function, which indicates that BOA is rotation and scale

Table 7
Simulated results of sine cosine algorithm (SCA).

Fun.	Affine Transformation									
	Original	Rotation	$Translation_1$	$Translation_2$	$Scale_1$	$Scale_2$	$General_1$	$General_2$	$General_3$	$General_4$
F1	1.57E-88	6.87E-02	6.14E + 00	1.50E + 02	5.77E-88	2.00E-85	9.79E-01	1.52E-01	1.51E + 02	1.01E + 02
F2	2.52E-58	1.75E + 05	1.09E + 01	2.93E + 10	1.43E-59	5.29E-60	1.62E + 07	8.56E + 03	3.89E + 10	2.89E + 09
F3	5.08E-04	9.14E + 03	1.11E + 01	2.24E + 04	2.34E-02	6.73E-03	9.23E + 03	8.28E + 03	2.30E + 04	1.33E + 04
F4	2.65E + 00	3.06E + 01	3.36E + 00	7.31E + 01	2.45E + 00	1.98E + 00	3.30E + 01	3.21E + 01	7.24E + 01	5.07E + 01
F5	3.31E-03	5.23E-01	2.14E + 01	9.76E + 01	3.12E-03	3.82E-03	1.81E + 00	5.69E-01	9.66E + 01	7.53E + 01
F6	2.31E-15	9.84E + 00	3.39E + 00	1.99E + 01	2.34E-15	2.27E-15	9.40E + 00	9.48E + 00	1.99E + 01	1.94E + 01
F7	3.29E-44	1.08E + 10	5.81E + 06	3.20E + 10	1.52E-44	6.35E-43	1.10E + 10	1.06E + 10	3.34E + 10	1.25E + 10
F8	3.71E + 10	1.22E + 10	3.67E + 10	3.87E + 10	3.51E + 10	3.63E + 10	1.25E + 10	1.24E + 10	4.01E + 10	1.38E + 10
F9	3.28E + 10	4.08E + 09	3.22E + 10	3.56E + 10	3.25E + 10	3.38E + 10	4.39E + 09	3.99E + 09	3.50E + 10	1.14E + 10
F10	6.75E + 01	6.35E + 01	6.61E + 01	9.49E + 01	6.75E + 01	6.71E + 01	6.52E + 01	6.56E + 01	8.96E + 01	7.13E + 01
F11	4.78E + 03	3.12E + 03	4.81E + 03	1.49E + 04	4.52E + 03	4.45E + 03	2.22E + 03	2.93E + 03	1.45E + 04	5.00E + 03
F12	2.08E + 03	1.83E + 03	2.11E + 03	2.74E + 03	2.05E + 03	2.18E + 03	1.84E + 03	1.84E + 03	2.66E + 03	2.06E + 03
F13	4.73E + 02	4.59E + 02	4.81E + 02	5.65E + 02	4.84E + 02	4.80E + 02	4.49E + 02	4.52E + 02	5.79E + 02	4.96E + 02
F14	5.98E + 03	4.10E + 03	5.93E + 03	5.69E + 03	5.86E + 03	5.86E + 03	4.00E + 03	4.05E + 03	5.57E + 03	4.50E + 03

invariant. But, results in other affine spaces are quite different from the original, which indicates that BOA is not translation and general invariant. Thus, the conclusion that BOA is not affine invariant is consistent with the conclusion shown in Section 2.5.6.

In Table 6 - Table 8, $scale_1$ and $scale_2$, $general_1$ and $general_2$, $general_3$ and $general_4$ have similar results in all functions, which indicate that the condition number of transformation matrix has no influence on the experimental results. $Translation_1$ and $translation_2$, $general_1$ and $general_3$, $general_2$ and $general_4$ have quite different results, which indicate that the value range of vector $\boldsymbol{b}$ have significant influence on the experimental results.

Fig. 2 shows the convergence curves of each algorithm in different affine spaces (the condition numbers of all affine transformation are 1) on F7. F7 is a large-scale function, which is difficult to be solved due to its high dimension. As shown in Fig. 2, the convergence curves of PSO, OFA and DE are overlapped together, which effectively indicate that PSO, OFA and DE have the same performance in all affine spaces, they are affine invariant. The convergence curves of GWO, SCA and BOA on F7 are not overlapped together, which indicate that they possess different performances in different affine spaces. From Fig. 2, it can be found that F7 can be solved by GWO and SCA within 1000 iterations in the original space and the scale affine space obtained, and BOA can solve F7 in the original space and the scale affine spaces and rotation affine space, which is a controversial result.

The results of numerical calculation and convergence curves analysis effectively support our theoretical analysis of the affine invariance of the algorithms, that is PSO, OFA and DE are affine invariant, GWO, SCA and BOA are not affine invariant.

4.3. Numerical statistical analysis

In order to analyze the difference of the results obtained by the algorithm in different affine spaces, variance of the data in Table 3-Table 8 are calculated and summarized in Table 9, and the box diagrams are drawn in Fig. 3. Variances represents the fluctuation degree of the data, which can be regard as a criterion of affine invariance. If the degree of fluctuation is small (variance is small), it means that the optimization results obtained by the algorithm in different affine spaces have little difference.

Table 8
Simulated results of butterfly optimization algorithm (BOA).

Fun.	Affine Transformation									
	Original	Rotation	$Translation_1$	$Translation_2$	$Scale_1$	$Scale_2$	$General_1$	$General_2$	$General_3$	$General_4$
F1	4.99E-17	4.99E-17	4.67E + 00	1.62E + 02	4.99E-17	4.99E-17	4.77E + 00	1.83E-01	1.62E + 02	1.52E + 02
F2	5.01E-14	5.01E-14	9.97E + 00	2.57E + 11	5.01E-14	5.01E-14	9.76E + 00	1.93E + 00	2.57E + 11	2.18E + 11
F3	6.81E-17	6.81E-17	1.12E + 01	3.66E + 04	6.81E-17	6.81E-17	1.18E + 01	5.08E-01	3.64E + 04	1.89E + 04
F4	5.31E-14	5.31E-14	8.07E-01	7.98E + 01	5.31E-14	5.31E-14	8.95E-01	1.46E-01	7.95E + 01	5.41E + 01
F5	7.35E-04	6.70E-04	7.12E + 00	9.91E + 01	7.12E-04	7.05E-04	7.25E + 00	5.09E-02	9.91E + 01	9.67E + 01
F6	2.93E-13	2.93E-13	2.99E + 00	2.00E + 01	2.93E-13	2.93E-13	3.05E + 00	5.33E-01	2.00E + 01	1.96E + 01
F7	8.14E-17	8.14E-17	5.89E + 06	5.56E + 10	8.14E-17	8.14E-17	6.28E + 06	2.60E + 04	5.52E + 10	1.04E + 10
F8	5.28E + 10	5.28E + 10	5.28E + 10	9.68E + 10	5.28E + 10	5.28E + 10	5.29E + 10	5.29E + 10	9.66E + 10	6.11E + 10
F9	5.24E + 10	5.24E + 10	5.25E + 10	1.11E + 11	5.24E + 10	5.24E + 10	5.23E + 10	5.23E + 10	1.11E + 11	7.92E + 10
F10	7.91E + 01	7.91E + 01	8.00E + 01	1.17E + 02	7.91E + 01	7.91E + 01	7.91E + 01	7.92E + 01	1.17E + 02	9.49E + 01
F11	4.86E + 03	4.86E + 03	4.75E + 03	2.42E + 04	4.86E + 03	4.86E + 03	4.84E + 03	4.80E + 03	2.48E + 04	1.05E + 04
F12	4.47E + 03	4.47E + 03	4.55E + 03	5.06E + 03	4.47E + 03	4.47E + 03	4.48E + 03	4.54E + 03	5.16E + 03	4.65E + 03
F13	3.81E + 02	3.81E + 02	3.94E + 02	7.35E + 02	3.81E + 02	3.81E + 02	3.52E + 02	3.73E + 02	7.41E + 02	6.28E + 02
F14	7.96E + 03	7.96E + 03	8.01E + 03	1.17E + 04	7.96E + 03	7.96E + 03	7.93E + 03	7.96E + 03	1.18E + 04	1.01E + 04

Table 9
Statistical variances of the results in different affine spaces.

Fun	Algorithm					
	PSO	OFA	DE	GWO	SCA	BOA
F1	2.43E-05	9.85E-25	1.42E-11	5.04E + 00	6.57E + 01	7.59E + 01
F2	1.65E-02	2.42E-13	4.66E-07	8.47E + 00	1.44E + 10	1.18E + 11
F3	2.27E-01	1.51E-11	9.54E-08	1.11E + 03	8.91E + 03	1.56E + 04
F4	5.60E-02	1.88E-02	1.01E-01	4.68E + 00	2.81E + 01	3.49E + 01
F5	1.94E-02	3.70E-04	1.10E-03	2.84E + 00	4.26E + 01	4.66E + 01
F6	3.08E-02	9.91E-13	3.79E-05	8.57E + 00	8.32E + 00	9.20E + 00
F7	7.05E + 06	1.24E-03	3.57E-01	1.73E + 09	1.26E + 10	2.30E + 10
F8	8.29E + 06	9.44E-02	2.20E + 05	2.64E + 09	1.28E + 10	1.82E + 10
F9	3.23E + 04	2.80E-01	1.58E + 05	7.45E + 08	1.45E + 10	2.47E + 10
F10	6.13E-01	2.44E-06	1.26E-03	1.85E + 00	1.10E + 01	1.59E + 01
F11	2.02E-01	1.48E-11	1.22E + 01	1.65E + 02	4.60E + 03	8.18E + 03
F12	4.04E + 00	8.51E-12	1.34E + 01	1.30E + 02	3.20E + 02	2.59E + 02
F13	7.02E-01	1.03E-11	2.10E-01	1.70E + 01	4.49E + 01	1.59E + 02
F14	2.88E + 01	5.58E-10	1.06E + 00	1.84E + 02	8.72E + 02	1.63E + 03

As shown in Table 9, the variances of PSO, OFA and DE are far less than the variances of GWO, SCA and BOA on most functions. The invariance of PSO on F7 is much larger than OFA and DE, but it is also much smaller than GWO, SCA and BOA. Similarly, the invariances of PSO and DE on F8 and F9 are much larger than OFA, but they are also much smaller than GWO, SCA and BOA. The variances of OFA on all function are much smaller that the others. The reason for this result is that the update method of OFA is spatial operation completely, while the update methods of PSO and DE include non-spatial operations, which intensifies the error accumulation in the process of affine transformation.

As shown in Fig. 3, the fluctuation degrees of the results of GWO, SCA and BOA are much higher than that of other algorithms on all functions. For F2, the fluctuation degree of the results of GWO seems to be the same as that of PSO, OFA and DE, but this is caused by the excessive coordinate axis. Combining with Table 9, it can be seen that the fluctuation degree of the results of GWO is much greater than that of PSO, OFA and DE.

The Kruskal-Wallis ANOVA test [15,40] is used to analyze the result of Table 9 to validate the invariance of these algorithm. The Kruskal-Wallis ANOVA table is shown in Table 10. The p-value ($p = 2.7E - 9$) is less than 0.05, which indicates that there are substantial differences among these algorithms. And then, a multiple comparison procedure is carried out to analyze the results of the Kruskal-Wallis test, and the estimated value of the mean ranks for each algorithm is achieved (shown in Table 11). As shown in Table 11, the estimated value of the mean ranks of OFA is the smallest, which means the degree of fluctuation of the results obtained by OFA in different affine spaces is the smallest. And DE and PSO are ranked second and third, respectively.

5. Discussion

5.1. Purpose of this work

The aim of this paper is to understand potential theoretical issues of the *meta*-heuristic algorithms and it does not try to improve the algorithms to find better solutions.

An algorithm whose performance relies on a privileged coordinate system is a poor choice in general. What is needed instead is a search algorithm that is affine invariant – one whose performance does not depend on the selection of the coordinate system. Affine invariance is a property of a swarm intelligence algorithm, which is used to judge whether the algorithm is influenced by the choice of coordinate systems or not.

6. Application limitations

The definition of affine invariance is based on the premise that the algorithm has spatial operations. In other words, non-spatial operations of the algorithm are ignored when defining its affine invariance, e.g. the crossover operation of DE. Affine invariance is non-exist if the algorithm has no spatial operations, e.g. GA. Most *meta*-heuristic algorithms possess spatial operations, which affect the performance of the algorithms. This kind of algorithms are concerned in this paper, including PSO, OFA, DE, GWO, SCA and BOA.

6.1. Condition number

The coordinate of the point $\boldsymbol{x}' = \boldsymbol{xA} + \boldsymbol{b}$ in affine space are not an exact value due to computer data overflow. After mapping back to the original space $\overline{\boldsymbol{x}} = (\boldsymbol{x}' - \boldsymbol{b})\boldsymbol{A}^{-1}$, the coordinate of $\boldsymbol{x}$ and $\overline{\boldsymbol{x}}$ might be very different if the matrix A has a very large condition number. The greater the condition number of $\boldsymbol{A}$, or the worse the accuracy of $\boldsymbol{x}'$, the greater the difference

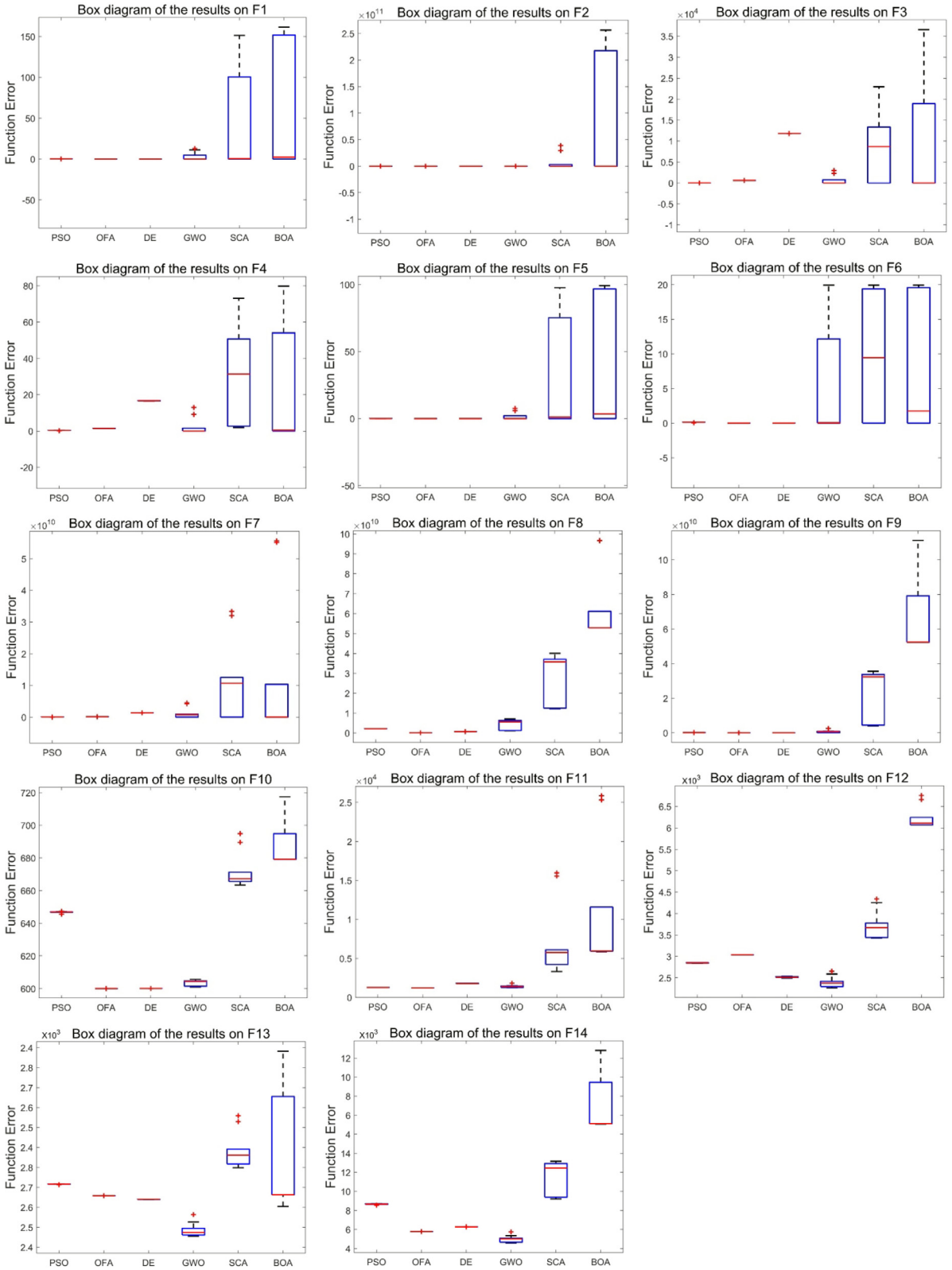

Fig. 3. The statistical box diagrams of the results in different affine spaces.

Table 10
Kruskal-Wallis ANOVA table.

Source	*SS*	*df*	*MS*	$Chi-sq$	$Prob > Chi-sq$
Groups	28906.3	5	5781.26	48.58	2.7e-09
Error	20478.7	78	262.55		
Total	49,385	839			

Table 11
The estimated values of the mean ranks.

	PSO	OFA	DE	GWO	SCA	BOA
Estimated value of the mean ranks	36.7143	11.1429	30	52.3571	61.1429	63.6429
Standard errors	6.5192	6.5192	6.5192	6.5192	6.5192	6.5192

between $\boldsymbol{x}$ and $\overline{\boldsymbol{x}}$. In order to avoid this situation, the data in the experiment is stored in double, and the condition number of $\boldsymbol{A}$ is in the range of [1,1000]. The experimental results show that the results have no obvious influence when the condition number of the matrix is 1000.

6.2. Computation complexity

There are two populations in the original space, and one of them is the mapping of population of affine space on the original space. Comparing with the algorithm evolution process on the original space, the algorithm evolution process on the affine space adds two parts: 1) in each iteration, mapping the population to the affine space; 2) in each iteration, mapping the updated individuals back to the original space to calculate their fitness values. Therefore, the added computational complexity is $2 \times t_{max} \times N \times D$, where t_{max} is the maximum number of iterations, N is the population size, D is the number of individual dimensions.

6.3. Multi-objective optimization algorithm

An important effect of affine invariance is to determine whether an algorithm is depended on a privileged coordinate system or not. An algorithm is sensitive to the representation of the optimal solution if it is not affine invariant. In multi-objective problems, the Pareto front is the optimal solution set. The core of multi-objective algorithm is the convergence and space requirement of their Pareto solution set. The convergence of the Pareto solution set is not guaranteed if the multi-objective algorithm is not affine invariant. For multi-objective algorithm, it can be converted to single objective algorithm to verify its affine invariance.

6.4. Future research suggestions

After reviewing the invariance of the search algorithms, there are several issues that we consider to be challenging and recommendations that the community working on swarm intelligence and bio-inspired algorithms should be dealt with in forthcoming years.

1) Researchers should focus on how to ensure the stability and reliability of the algorithm. The current literature is flooded with a huge number of bio-inspired algorithms, however, there is no significant breakthrough in solving the optimization problems.
2) Only few algorithms have a strong influence and their potential properties are explored deeply, e.g. DE, PSO, GA. Researchers should explore other novel algorithms to obtain new inspiration, so as to enrich and develop the community of optimization algorithms.
3) In the experiments of algorithm comparison, researchers should analyze the reason of why the proposed algorithm performs poorly in some functions, so as to obtain the prior knowledge of how to improve the defects.

7. Conclusions

Invariance is an important aspect in continuous domain search. The most importance invariances are reviewed. Invariances induce equivalence classes of objective functions and consequently guarantee the generalization of performance results within each class, thereby considerably strengthening their relevance. Invariance generalizes any result and is not a priori associated with sound performance.

According to the practical implementation of the algorithm, we believe that the invariant algorithm does not mean that it should maintain invariant during the whole search process, but retain invariant when executing the operation in space. To

achieve this goal, the definition of affine invariance is provided in this paper. An algorithm is independent of the coordinate system if it is affine invariant, which means that the algorithm has the same performance in different affine spaces.

In this paper, the affine invariances of six algorithms are analyzed. As a conclusion, PSO, OFA and DE are theoretically and experimental proved to be affine invariant, while GWO, SCA and BOA are not. However, can we conclude that a good algorithm must be affine invariant? The answer is not. First, not all *meta*-heuristic algorithms possess the property of affine invariance. Affine invariance is defined under the operation of space, and some algorithms are not executed in space, e.g. GA. Second, even the algorithm is not affine invariant, it can still solve various optimization problems efficiently if its operation in space is not important. Finally, for random search algorithms, experiment is the most important method to test their performances, and the theoretical analysis and experimental results should corroborate each other.

CRediT authorship contribution statement

ZhongQuan Jian: Conceptualization, Software, Writing - original draft, Formal analysis, Resources. **GuangYu Zhu:** Methodology, Funding acquisition, Project administration, Writing - review & editing.

Declaration of Competing Interest

We declare that we have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Intelligent Manufacturing Integrated Standardization and New Model Application Project in 2016 of MIIT, under Grant (2016) 213.

Appendix A. Supplementary data

Supplementary data to this article can be found online at https://doi.org/10.1016/j.ins.2021.06.062.

References

[1] F.S. Gharehchopogh, H. Shayanfar, H. Gholizadeh, A comprehensive survey on symbiotic organisms search algorithms, Artif. Intell. Rev. 53 (3) (2020) 2265–2312.
[2] A.E. Eiben, J. Smith, From evolutionary computation to the evolution of things, Nature. 521 (7553) (2015) 476–482.
[3] C. Blum, R. Chiong, M. Clerc, K. De Jong, Z. Michalewicz, F. Neri, T. Weise, in: Variants of Evolutionary Algorithms for Real-World Applications, Springer Berlin Heidelberg, Berlin, Heidelberg, 2012, pp. 1–29, https://doi.org/10.1007/978-3-642-23424-8_1.
[4] R. Rajakumar, P. Dhavachelvan, T. Vengattaraman, A survey on nature inspired meta-heuristic algorithms with its domain specifications, In: 2016 International Conference on Communication and Electronics Systems (ICCES), 2017.
[5] D. Karaboga, B. Gorkemli, C. Ozturk, N. Karaboga, A comprehensive survey: artificial bee colony (ABC) algorithm and applications, Artif. Intell. Rev. 42 (1) (2014) 21–57.
[6] X.S. Yang, S. Deb, Cuckoo search: recent advances and applications, Neural Comput. Appl. 24 (1) (2014) 169–174.
[7] S. Das, S.S. Mullick, P.N. Suganthan, Recent advances in differential evolution – an updated survey, Swarm Evol. Comput. 27 (2016) 1–30.
[8] S. Das, P.N. Suganthan, Differential evolution: a survey of the state-of-the-art, IEEE Trans. Evol. Comput. 15 (1) (2011) 4–31.
[9] F. Neri, V. Tirronen, Recent advances in differential evolution: a review and experimental analysis, Artif. Intell. Rev. 33 (1) (2010) 61–106.
[10] R.D. Al-Dabbagh, F. Neri, N. Idris, M.S. Baba, Algorithm design issues in adaptive differential evolution: review and taxonomy, Swarm Evol. Comput. 43 (2018) 284–311.
[11] I. Fister jr, M. Perc, S. Kamal, I. Fister, A review of chaos-based firefly algorithms: perspectives and research challenges, Appl. Math. Comput. 252 (2015) 155–165.
[12] Yingying Song, Fulin Wang, Xinxin Chen, An improved genetic algorithm for numerical function optimization, Appl. Intell. 49 (5) (2019) 1880–1902.
[13] Mohammad Reza Bonyadi, Zbigniew Michalewicz, Particle swarm optimization for single objective continuous space problems: a review, Evol. Comput. 25 (1) (2017) 1–54.
[14] L. Juan, X.J. Wu, V. Palade, W. Fang, Y.H. Shi, Random drift particle swarm optimization algorithm: convergence analysis and parameter selection, Mach. Learn. 101 (1–3) (2015) 345–376.
[15] Guang-Yu Zhu, Wei-Bo Zhang, Optimal foraging algorithm for global optimization, Appl. Soft. Comput. 51 (2017) 294–313.
[16] H. Faris, I. Aljarah, M.A. Al-Betar, S. Mirjalili, Grey wolf optimizer: a review of recent variants and applications, Neural. Comput. Appl. 30 (22) (2018) 413–435.
[17] S. Mirjalili, SCA: a sine cosine algorithm for solving optimization problems, Knowl-Based. Syst. 96 (2016) 120–133.
[18] S. Arora, S. Singh, Butterfly optimization algorithm: a novel approach for global optimization, Soft Comput. 23 (3) (2019) 715–734.
[19] D. Molina, J. Poyatos, J. Del Ser, S. García, A. Hussain, F. Herrera, Comprehensive taxonomies of nature- and bio-inspired optimization: inspiration versus algorithmic behavior, critical analysis recommendations, Cogn. Comput. 12 (1) (2020) 897–939.
[20] N. Hansen, R. Ros, N. Mauny, M. Schoenauer, A. Auger, Impacts of invariance in search: When CMA-ES and PSO face ill-conditioned and non-separable problems, Appl. Soft Comput. 11 (8) (2011) 5755–5769.
[21] N. Hansen, A. Auger, CMA-ES, evolution strategies and covariance matrix adaptation, in: In: Proceedings of the 13th annual conference companion on Genetic and evolutionary computation, 2011, pp. 991–1010.
[22] D. Wilke, S. Kok, A. Groenwold, Comparison of linear and classical velocity update rules in particle swarm optimization: Notes on scale and frame invariance, Int. J. Numer. Meth. Eng. 70 (8) (2007) 985–1008.
[23] M.R. Bonyadi, Z. Michalewicz, SPSO2011 – analysis of stability, local convergence, and rotation sensitivity, Genetic Evol. Comput. Conf. (GECCO) (2014) 9–15.

[24] Mohammad Reza Bonyadi, Zbigniew Michalewicz, A locally convergent rotationally invariant particle swarm optimization algorithm, Swarm. Intell. 8 (3) (2014) 159–198.
[25] Fabio Caraffini, Ferrante Neri, A study on rotation invariance in differential evolution, Swarm. Evol. Comput. 50 (2019) 100436, https://doi.org/10.1016/j.swevo.2018.08.013.
[26] K.V. Price, R.N. Storn, J. Lampinen, Differential Evolution: A Practical Approach to Global Optimization, Springer, 2005.
[27] J.R. Kirkwood, B.H. Kirkwood, Elementary Linear Algebra, (1st ed.),., Chapman and Hall/CRC, 2018.
[28] M. K. Murray, Differential geometry and statistics, London, UK: Routledge, 2017.
[29] Ferrante Neri (Ed.), Linear Algebra for Computational Sciences and Engineering, Springer International Publishing, Cham, 2019.
[30] F. Neri, Y.Y. Zhou, Covariance Local Search for Memetic Frameworks: A Fitness Landscape Analysis Approach, in, IEEE Congress on Evolutionary Computation (CEC) 2020 (2020) 1–8.
[31] F. Neri, S. Rostami, A Local Search for Numerical Optimisation based on Covariance Matrix Diagonalisation, In: Castillo P.A., Jiménez Laredo J.L., Fernández de Vega F. (eds), Applications of Evolutionary Computation, EvoApplications 2020, Lecture Notes in Computer Science, vol. 12104, Springer, Cham, 2020, pp. 3-19.
[32] F. Neri, Adaptive Covariance Pattern Search, In: Castillo P.A., Jiménez Laredo J.L. (eds), Applications of Evolutionary Computation, EvoApplications 2021, Lecture Notes in Computer Science, vol. 12694, Springer, Cham, 2021, pp. 178-193.
[33] F. Neri, S. Rostami, Generalised pattern search based on covariance matrix diagonalisation, SN Comput. Sci. 2 (3) (2021) 171.
[34] Elena Anne Corie Marchisotto, Francisco Rodríguez-Consuegra, James T. Smith, in: The Legacy of Mario Pieri in Foundations and Philosophy of Mathematics, Springer New York, New York, NY, 2021, pp. 223–244, https://doi.org/10.1007/978-0-8176-4823-7_7.
[35] Zizhong Chen, Jack J. Dongarra, Condition numbers of gaussian random matrices, SIMAX. 27 (3) (2005) 603–620.
[36] Charles Kenney, Alan J. Laub, Condition estimates for matrix functions, SIMAX. 10 (2) (1989) 191–209.
[37] V.S. Antyufeev, Probabilistic estimation of matrix condition number, J. Math. Sci. 246 (6) (2020) 755–762.
[38] M. Jamil, X.S. Yang, A literature survey of benchmark functions for global optimization problems, IJMMNO. 4 (2) (2013) 150–194.
[39] N. H. Awad, M. Z. Ali, P. N. Suganthan, J. J. Liang and B. Y. Qu, Problem Definitions and Evaluation Criteria for the CEC 2017 Special Session and Competition on Single Objective Real-Parameter Numerical Optimization, in: Nanyang Technological University, Jordan University of Science and Technology and Zhengzhou University, Tech. Rep. 2016.
[40] J. Richardson, Kruskal-Wallis Test, SAGE Publications (2018) 937–939.